

2024 - 1º fecha

1.

a. ¿Cuándo un lenguaje es recursivamente enumerable y cuándo un lenguaje es recursivo?

Un lenguaje es recursivamente enumerable cuando existe una MT que acepta todas las cadenas del lenguaje pero puede no detenerse para cadenas que no pertenecen al lenguaje.

Un lenguaje es recursivo cuando existe una MT que acepta todas las cadenas del lenguaje y siempre se detiene.

b. ¿Existen lenguajes recursivamente enumerables no recursivos? Dar un ejemplo.

Sí, existen lenguajes que están en $RE - R$, es decir son recursivamente enumerables no recursivos. El ejemplo típico es HP .

c. ¿Existen lenguajes no recursivamente enumerables? Dar un ejemplo.

Sí, existen lenguajes que no poseen ninguna MT que siempre se detenga ni siquiera para todas las cadenas que sí pertenecen al lenguaje. Ejemplo: $HP_{total} = \{\langle M \rangle \mid M \text{ se detiene para todas las cadenas}\}$

2. Construir (dar sólo la idea general) una MT que acepte los pares de códigos de MT $(\langle M_1 \rangle, \langle M_2 \rangle)$ tales que la MT M_1 o la MT M_2 acepten la cadena vacía.

...

3. Se prueba que todos los lenguajes recursivamente enumerables se reducen al lenguaje HP , representante del problema de la detención. Explicar por qué, si HP fuera recursivo, también lo serían todos los lenguajes recursivamente enumerables.

...

4. Justificar por qué la siguiente función f no es una reducción del lenguaje HP al lenguaje HP^C (el complemento de HP): a todo par $(\langle M_1 \rangle, w)$, f le asigna el par $(\langle M_2 \rangle, w)$, tal que $\langle M_2 \rangle$ es como $\langle M_1 \rangle$ salvo que sus estados finales están invertidos (es decir, donde aparece q_A en $\langle M_1 \rangle$ aparece q_R en $\langle M_2 \rangle$ y viceversa). Comentario: asumir que los pares $(\langle M_1 \rangle, w)$ siempre son sintácticamente correctos.

...

5.

a. ¿Cuándo un lenguaje pertenece a la clase P y cuándo un lenguaje pertenece a la clase NP ?

...

b. Dar un ejemplo de lenguaje en NP que no estaría en P , y justificar por qué no estaría en P .

...

c. Dar un ejemplo de lenguaje que no estaría en NP , y justificar por qué no estaría en NP .

...

6. Describir el método que estudiamos para poblar la clase de los lenguajes NPC , contando ya al menos con un lenguaje que pertenezca a dicha clase, y justificar por qué es correcto.

...

7.

a. ¿Qué lenguajes se consideran tratables en la jerarquía espacial? Justificar por qué.

...

b. Explicar por qué una MT que ocupa espacio $poly(n)$ puede tardar más que tiempo $poly(n)$.

...

c. Dada una MT M_1 que ocupa espacio $S(n)$, explicar cómo se puede construir a partir de ella una MT M_2 que en espacio $S(n)$ imprima una cadena con $S(n)$ marcas X .

...

8.

a. Especificar un programa que calcule el mcd , es decir el máximo común divisor de dos números enteros. Comentario: en la postcondición se puede usar la notación $mcd(x, y)$.

Asumo que por especificación solo se pide dar la terna de Hoare $\{p\}S\{q\}$ sin necesidad de definir explícitamente S .

Una terna posible podría ser $\{x = X \wedge y = Y\}S\{z = mcd(X, Y)\}$

b. Justificar por qué $(x > 0, y = x)$ no es una especificación correcta de un programa que debe devolver, en la variable de salida y , el valor de la variable de entrada x .

La especificación no es correcta porque no te asegura que la variable y tenga el valor que tenía x al iniciar el programa, ya que x podría haber sido modificado luego de haber iniciado el programa. Se deberían usar variables lógicas.

c. El hecho de que un programa satisfaga la especificación $(x > 0, x > 0)$, ¿asegura que la aserción $x > 0$ se cumpla a lo largo de toda su ejecución? Justificar la respuesta.

No, no asegura la aserción. Podría pasar que $x = 8$ al inicio del programa, luego se ejecute la instrucción $x = -5$ y finalmente, antes de terminar el programa, se ejecute la instrucción $x = 2$. Si bien se cumple que $x > 0$ tanto antes de iniciar el programa como después de terminada su ejecución, no se cumplió a lo largo de TODA su ejecución.

9. Asumiendo que se cumple la fórmula de correctitud total $\langle p \rangle S \langle q \rangle$ indicar, justificando la respuesta, si se cumple cada uno de los siguientes enunciados:

a. Si S no termina, entonces su estado inicial no satisface p .

Por contrapositiva, el enunciado es equivalente a decir: si el estado inicial satisface p , entonces S termina. Esto es **verdadero** por definición de correctitud total: si p se cumple entonces necesariamente S termina en un estado que satisface q .

b. Si S termina en un estado que satisface q , entonces su estado inicial satisface p .

Nuevamente, por contrapositiva, el enunciado es equivalente a decir: si el estado inicial no satisface p , entonces S no termina, o termina en un estado que no satisface q . Esto es **falso**, puede ocurrir que S termine en un estado que sí satisface q , o incluso que nunca termine. Como ya la precondición no se cumple, todo lo demás es irrelevante.

c. Si S termina en un estado que no satisface q , entonces su estado inicial no satisface p .

Contrapositiva: si el estado inicial satisface p , entonces S no termina, o termina en un estado que satisface q . Esto necesariamente es **verdadero**, ya que el consecuente posee una disyunción que solo es falsa cuando todas sus condiciones son falsas, y si analizamos lo que tenemos, es imposible que esto suceda. Si S no termina, claramente no termina en un estado que satisface q . Si S termina en un estado que satisface q , claramente sí terminó. Por lo tanto, suceda lo que suceda, la implicación es verdadera.

10. Probar mediante el método H , usando el invariante p , la fórmula de correctitud parcial $\{p\}\text{while } \neg q \text{ do skip od}\{q\}$.

- i. Precondición: p
 - ii. Postcondición: q
 - iii. Condición del while: $\neg q$
 - iv. Invariante: p
2. Probar que la precondición implica al invariante antes de entrar al while:
- i. $p \rightarrow p$
 - ii. Se cumple trivialmente, toda variable proposicional se implica a sí misma.
3. Probar que el invariante se preserva en cada iteración:
- i. $\{invariante \wedge condición\} cuerpo \{invariante\}$
 - ii. $\{p \wedge \neg q\} skip \{p\}$
 - iii. Se cumple trivialmente ya que $(p \wedge \neg q) \rightarrow p$ es una tautología.
4. Probar la salida del while:
- i. $invariante \wedge \neg condición \rightarrow postcondición$
 - ii. $(p \wedge \neg(\neg q)) \rightarrow q$
 - iii. $(p \wedge q) \rightarrow q$
 - iv. Lo anterior se cumple por ser una tautología: la única forma de que la fórmula sea falsa es que el antecedente sea V y el consecuente F . Pero si $q = F$, que es la única forma de que el consecuente sea falso, entonces el antecedente también es falso por tener un AND lógico. Por lo tanto la fórmula se cumple siempre.

2025 - Ejemplo de examen

1. Probar que el conjunto RE es cerrado con respecto a la operación de intersección $\cap$, es decir que para todo par de lenguajes $L_1, L_2 \in RE$, se cumple que $L_1 \cap L_2 \in RE$.

Para probar esto podemos construir una MT M que reconozca $L_1 \cap L_2$ a partir de las MT M_1 y M_2 que reconocen L_1 y L_2 , respectivamente.

M hace lo siguiente:

- 1. Recibe una cadena de entrada w .
- 2. Simula a M_1 a partir de w :
 - i. Si M_1 acepta, se pasa al paso 3.
 - ii. Si M_1 rechaza, M rechaza.
 - iii. Si M_1 loopea, M loopea.
- 3. Simula a M_2 a partir de w :
 - i. Si M_2 acepta, M acepta.
 - ii. Si M_2 rechaza, M rechaza.
 - iii. Si M_2 loopea, M loopea.

Verificando el funcionamiento de M :

- 1. Si $w \in L_1 \cap L_2$:
 - i. M_1 acepta w , entonces se pasa al paso 3.
 - ii. M_2 acepta w , entonces M acepta w .
 - iii. Por lo tanto, M acepta w .
- 2. Si $w \notin L_1 \cap L_2$:
 - i. M_1 rechaza o loopea
 - ii. O M_1 acepta pero M_2 rechaza o loopea.
 - iii. En ambos casos, M no acepta w .

Por lo tanto M reconoce $L_1 \cap L_2$, y por lo tanto $L_1 \cap L_2 \in RE$.

2. Sea L_1 un lenguaje generado con los símbolos del alfabeto Σ_1 . Sea $f : \Sigma_1 \rightarrow \Sigma_2^*$ una función total computable, consistente en asignar a cada símbolo de Σ_1 una cadena de Σ_2^* . Y sea L_2 el lenguaje formado por las cadenas obtenidas de la aplicación de f sobre las cadenas de L_1 . Es decir que dado $L_1 = \{w_1, w_2, \dots\}$, el lenguaje L_2 es $\{f(w_1), f(w_2), \dots\}$, entendiendo que $f(w)$, con $w = w_1 w_2 \dots w_n$ significa $f(w_1) f(w_2) \dots f(w_n)$. Por ejemplo, si $abc \in L_1$, entonces $f(a) = 1$,

$f(b) = 00$ y $f(c) = 101$, entonces $f(abc) = 100101 \in L_2$. Probar que si $L_1 \in R$, entonces $L_2 \in RE$. Ayuda: Construir una MT que reconozca L_2 .

...

3. El problema K-CLIQUE consiste en determinar si un grafo no orientado incluye un clique de tamaño k , es decir un subgrafo completo de k vértices. El lenguaje que lo representa es $K\text{-CLIQUE} = \{(G, k) \mid G \text{ es un grafo no orientado, } k > 0 \in \mathbb{N}, G \text{ incluye un clique de tamaño } k\}$. Probar que $3\text{-CLIQUE} \in P$, es decir, se puede determinar en tiempo determinístico polinomial si un grafo no orientado incluye un triángulo (clique de tamaño 3).

...

4. El problema PARTICIÓN consiste en determinar si un conjunto finito de números naturales se puede partir en dos conjuntos C_1 y C_2 , tales que la suma de los números de C_1 sea igual a la suma de los números de C_2

...

5. La sintaxis de un determinado lenguaje de programación L se define de la siguiente manera. Un programa S pertenece a L si y sólo si S tiene la forma:

$$S :: \text{skip} \mid x := e \mid S_1; S_2 \mid \text{case of } B_1 \rightarrow S_1; B_2 \rightarrow S_2; \text{otherwise} \rightarrow S_3 \text{ end}$$

$$e :: n \mid x \mid e_1 + e_2$$

$$B ::= \text{true} \mid \text{false} \mid e_1 = e_2 \mid \neg B$$

donde e es una expresión entera, n es una constante entera, x es una variable entera, y B es una expresión booleana. Sea $\text{var}(S)$ la expresión que denota el conjunto de las variables que aparecen en el programa S . Definir por inducción el conjunto $\text{var}(S)$.

- Expresiones enteras:
 - $\text{var}(n) = \emptyset$ porque n no es una variable.
 - $\text{var}(x) = \{x\}$ porque x es una variable.
 - $\text{var}(e_1 + e_2) = \text{var}(e_1) \cup \text{var}(e_2)$
- Expresiones booleanas:
 - $\text{var}(\text{true}) = \emptyset$
 - $\text{var}(\text{false}) = \emptyset$
 - $\text{var}(e_1 = e_2) = \text{var}(e_1) \cup \text{var}(e_2)$
 - $\text{var}(\neg B) = \text{var}(B)$
- Programas:
 - $\text{var}(\text{skip}) = \emptyset$
 - $\text{var}(x := e) = \{x\} \cup \text{var}(e)$
 - $\text{var}(S_1; S_2) = \text{var}(S_1) \cup \text{var}(S_2)$
 - $\text{var}(\text{case of } B_1 \rightarrow S_1; B_2 \rightarrow S_2; \text{otherwise} \rightarrow S_3 \text{ end}) = \text{var}(B_1) \cup \text{var}(S_1) \cup \text{var}(B_2) \cup \text{var}(S_2) \cup \text{var}(S_3)$

6. Probar empleando el método de verificación H :

a. $\{p\} \text{while } \text{true} \text{ do skip od} \{q\}$

1. Se elige como invariante a p .
2. $\{p \wedge \text{true}\} \text{skip} \{p\}$
3. $\{p\} \text{skip} \{p\}$ (una variable y verdadero siempre es equivalente a la variable)

4. Lo anterior es válido por la regla del skip.
5. $\{p\}\text{while } true \text{ do skip od}\{p \wedge \neg true\}$ (aplicación de la regla del while)
6. $\{p\}\text{while } true \text{ do skip od}\{p \wedge false\}$
7. $\{p\}\text{while } true \text{ do skip od}\{false\}$
8. Dado que $false \Rightarrow q$, por regla de consecuencia reemplazamos $false$ por q .
9. $\{p\}\text{while } true \text{ do skip od}\{q\}$

b. $\{p\}\text{while } \neg q \text{ do skip od}\{q\}$

Ya resuelto en el parcial anterior.

2025 - 1º fecha

1. Definir cuándo un lenguaje pertenece a las siguientes clases y además dar un ejemplo de lenguaje de cada clase:

a. R

Un lenguaje pertenece a R si existe una MT que lo decide y siempre se detiene.

Ejemplo: Σ^*

b. $RE - R$

Un lenguaje pertenece a $RE - R$ si existe una MT que lo acepta pero puede looppear para cadenas que no pertenecen al lenguaje.

Ejemplo: HP

c. $CO-RE - R$

Un lenguaje pertenece a $CO-RE - R$ si su complemento está en RE pero no es un lenguaje decidable.

Ejemplo: HP^C

d. $\mathcal{L} - (RE \cup CO-RE)$

Un lenguaje pertenece a $\mathcal{L} - (RE \cup CO-RE)$ si no existe ninguna MT que lo acepte ni tampoco a su complemento.

Ejemplo: $HP_{total} = \{\langle M \rangle \mid M \text{ se detiene para toda entrada}\}$

2. Sea $L_1 \in R$ y $L_2 \in RE$.

a. Probar, construyendo una MT M que $L_1 \cup L_2 \in RE$. Solo dar, claramente, la idea general de M .

1. Sean M_1 y M_2 las MT que aceptan a L_1 y a L_2 respectivamente.
2. M recibe una cadena de entrada w .
3. M simula a M_1 a partir de w . Como $L_1 \in R$, M_1 nunca looppea.
 - i. Si M_1 acepta, M acepta.
 - ii. Si M_1 rechaza, se pasa al paso 4.
4. M simula a M_2 a partir de w :
 - i. Si M_2 acepta, M acepta.
 - ii. Si M_2 rechaza, M rechaza.
 - iii. Si M_2 looppea, M looppea.

b. Dar un ejemplo de lenguaje L_1 de R y un lenguaje L_2 de RE tal que $L_1 \cup L_2 \notin R$.

$$L_1 = \{0, 00, 11\}$$

$$L_2 = HP$$

$$L_1 \cup L_2 \notin R$$

3. En el marco de la computabilidad:

a. ¿Qué significa que una prueba sea constructiva?

En la computabilidad, una prueba es constructiva cuando se prueba vía la construcción de una máquina de Turing que cumple un determinado objetivo.

b. ¿Por qué las pruebas de no pertenencia a R o a RE no pueden ser constructivas?

Para probar que un lenguaje no está en R o en RE se debe probar que no existe **ninguna** MT que lo decida o que lo reconozca, respectivamente. Justamente, como queremos probar que no hay ninguna MT no podemos construir una MT que lo pruebe. No se puede construir una MT que pruebe que ninguna MT (que son infinitas) puede hacer determinada cosa.

c. ¿Qué técnicas existen para probar la no pertenencia a R o a RE ?

Existen dos técnicas principales, la diagonalización y la reducción.

4. Indicar, justificando la respuesta, si se cumplen los siguientes enunciados, todos referidos al lenguaje HP :

a. Si $L \notin RE$, entonces no existe una reducción de L a HP .

Verdadero.

Por teorema se sabe que si $L_1 \leq L_2$ y $L_2 \in RE$, entonces $L_1 \in RE$. Como se sabe que $HP \in RE$, L debe ser RE pero no lo es. Contradicción. Por lo tanto no existe reducción de L a HP .

b. Si existe una reducción de HP a un lenguaje L , entonces $L \in RE$.

Falso.

Es posible reducir HP a un lenguaje que no está en RE , como por ejemplo $HP_{total} = \{\langle M \rangle \mid M \text{ se detiene para toda entrada}\}$

c. Si todos los lenguajes de R se reducen a HP , entonces $R = RE$.

Falso.

Ya de por sí es verdad que todos los lenguajes de R se pueden reducir a HP , y sin embargo se sabe que $R \neq RE$.

5. Definir cuándo un lenguaje (además, dar un ejemplo de lenguaje de cada clase):

a. Pertenecer a la clase P .

Un lenguaje pertenece a la clase P cuando existe una MT que lo decide en tiempo polinomial.

Ejemplo: $\{00, 01, 11\}$

b. Pertenecer a la clase NP .

Un lenguaje pertenece a la clase NP cuando existe una MT y un certificado tal que la MT decide al lenguaje en tiempo polinomial usando ese certificado como atajo.

Ejemplo: SAT

c. Pertenecer a la clase NPI .

Un lenguaje pertenece a la clase NPI cuando está en NP , no está en P , y no es NPC .

Ejemplo: ISO , el problema de los grafos isomorfos.

6. Probar la pertenencia a la clase NP del lenguaje $PARTICIÓN$, definido como el conjunto C de números naturales que se puede particionar en dos subconjuntos C_1 y C_2 tales que la suma de los

elementos de C_1 coincide con la suma de los elementos de C_2 . Comentario: se sabe que la suma de k números se puede realizar en tiempo polinomial.

Ejemplo: $C = \{1, 2, 3, 4\}$ se puede particionar en $C_1 = \{1, 4\}$ y $C_2 = \{2, 3\}$, ya que $1 + 4 = 2 + 3$.

Para esto, se debe construir una MT M que, a partir de un certificado x , decida *PARTICIÓN* en tiempo polinomial.

M recibe como cadena de entrada a un conjunto de números naturales C y a un certificado x que es una lista de ceros y unos donde un cero significa que el número correspondiente de C pertenece a C_1 y un uno significa que el número correspondiente de C pertenece a C_2 .

Por ejemplo, si $C = \{1, 2, 3, 4\}$ y $x = 0110$, entonces $C_1 = \{1, 4\}$ y $C_2 = \{2, 3\}$.

M recorre de forma lineal tanto el certificado como el conjunto C y va sumando los números de C_1 y los números de C_2 , cosa que se sabe por enunciado que es polinomial. Al finalizar, M compara ambas sumas. Si son iguales, acepta. Si no, rechaza.

El tiempo de M es $O(n) + O(n) + O(1) = O(n)$, donde n es el tamaño de la entrada, es decir el tamaño del conjunto C .

M decide *PARTICIÓN* en tiempo polinomial usando un certificado, y por lo tanto *PARTICIÓN* $\in NP$.

7. Indicar, justificando la respuesta, si se cumplen los siguientes enunciados, todos referidos al lenguaje *SAT* (el lenguaje del problema de satisfactibilidad):

a. Todos los lenguajes de P se reducen polinomialmente a *SAT*.

Verdadero.

Se sabe que *SAT* es un lenguaje *NPC*, y por definición de *NPC* se sabe que todos los lenguajes de *NP* se reducen polinomialmente a *SAT*. Como P está contenido en *NP*, entonces todos los lenguajes de P se reducen polinomialmente a *SAT*.

b. Si existe una reducción polinomial de *SAT* a un lenguaje L , entonces $L \in NPC$.

Falso.

Puede pasar que L sea *NPH* pero no *NPC*, como por ejemplo *HP*.

c. Si existe una reducción polinomial de *SAT* a un lenguaje de P , entonces $P = NP$.

Verdadero.

Por teorema, se sabe que si $L_1 \leq L_2$ y $L_2 \in P$, entonces $L_1 \in P$. Como $SAT \leq_p L$ y $L \in P$, entonces $SAT \in P$. Como *SAT* es *NPC*, todo lenguaje de *NP* se reduce polinomialmente a *SAT*, y como $SAT \in P$, todo lenguaje de *NP* se reduce polinomialmente a un lenguaje de P . Por lo tanto no solo se cumple $P \subseteq NP$, sino que también se cumple $NP \subseteq P$. Por lo tanto $P = NP$.

8. Responder:

a. ¿Cuándo un programa es correcto parcialmente con respecto a una especificación?

Un programa es correcto **parcialmente** con respecto a una especificación cuando dada una precondition y una postcondition, si se cumple la precondition y el programa termina entonces la postcondition se cumple.

b. ¿Cuándo un programa es correcto totalmente con respecto a una especificación?

Un programa es correcto **totalmente** con respecto a una especificación cuando dada una precondition y una postcondition, si se cumple la precondition entonces el programa termina y la postcondition se cumple.

c. ¿Por qué se hace la distinción entre correctitud parcial y correctitud total?

La distinción se hace primordialmente para diferenciar a los programas que siempre terminan (totalmente correctos) vs los que **pueden** no terminar (parcialmente correctos).

9. Especificar un programa que calcule la raíz cuadrada de un número x que sea entero y mayor estricto que cero, y tal que al final, x tenga el mismo valor que al inicio del programa.

$$\{x = X \wedge x \in \mathbb{Z} \wedge x > 0\} S \{y = \sqrt{X} \wedge x = X\}$$

10. Probar con el método H :

a. $\{x = X \wedge X > 0\}y := 2x\{x = X \wedge y \geq 2\}$.

1. $\{x = X \wedge X > 0\}y := 2x\{x = X \wedge y \geq 2\}$
2. $\{x = X \wedge 2x \geq 2\}y := 2x\{x = X \wedge y \geq 2\}$ (ASI)
3. $(x = X \wedge X > 0) \Rightarrow (x = X \wedge 2x \geq 2)$?
4. $(x = X \wedge X > 0) \Rightarrow (x > 0) \Rightarrow (x \geq 1) \Rightarrow (2x \geq 2) \Rightarrow (x = X \wedge 2x \geq 2)$
5. Por lo tanto, por regla CONS, la precondition original $x = X \wedge X > 0$ implica a la obtenida $(x = X \wedge 2x \geq 2)$ y por ende la terna es válida.

b. $\{true\}\text{while true do skip od}\{false\}$

1. Tomamos como invariante a la misma precondition: $i = true$
2. $\{true \wedge true\}\text{skip}\{true\}$ (REP)
3. $\{true\}\text{skip}\{true\}$ (lógica)
4. $\{true\}\text{while true do skip od}\{true \wedge \neg true\}$ (REP)
5. $\{true\}\text{while true do skip od}\{true \wedge false\}$ (lógica)
6. $\{true\}\text{while true do skip od}\{false\}$ (lógica)

Desconocido

1. Sean $L_1 \in RE$ y $L_2 \in R$.

a. Probar, construyendo una MT, que $L_1 \cap L_2 \in RE$. Comentario: sólo se pide presentar la idea general de la MT.

...

b. Encontrar un lenguaje $L_1 \in RE$ y un lenguaje $L_2 \in R$ tales que $L_1 \cap L_2 \notin R$.

...

2. Construir una MT que acepte el lenguaje $L = \{(\langle M \rangle, w_1, w_2) \mid \text{la MT } M \text{ acepta al menos una de las cadenas } w_i\}$. Comentario: sólo se pide presentar la idea general de la MT.

...

3. La siguiente es una prueba de que $L_U \notin R$

a. Existe un lenguaje L tal que $L^C \notin RE$ y además $L \leq L_U$.

...

b. Como $L^C \notin RE$, entonces $L \notin R$.

...

c. Y como $L \notin R$ y $L \leq L_U$, entonces $L_U \notin R$.

...

Se pide justificar los incisos b y c. Ayuda: tener en cuenta las propiedades estudiadas en clase sobre las clases R y RE y sobre las reducciones de lenguajes.

...

4. Probar que si A es Turing-reducible a B , entonces A^C es Turing-reducible a B^C . Ayuda: teniendo en cuenta la hipótesis, hay que probar que existe una MT con oráculo B^C que acepta A^C y siempre se detiene.

...

5. Probar que cualquier lenguaje $L_1 \in P$ se puede reducir polinomialmente a cualquier lenguaje $L_2 \in P$, siempre y cuando L_1 y L_2 no sean ni Σ^* ni $\emptyset$. Comentario: describir la función de reducción de L_1 a L_2 y probar que la función se computa en tiempo polinomial.

...

6. Construir una MT que acepte todos los pares $(\langle M \rangle, n)$ siendo n un número natural, que cumplen que M , a partir del input vacío, se detiene en a lo sumo n^2 pasos. Ayuda: la función n^2 es tiempo-construible. Comentario: sólo presentar la idea general de la MT.

...

7. Probar que el siguiente lenguaje está en NP : $\text{TRUE-SAT} = \{\phi \mid \phi \text{ es una fórmula booleana sin cuantificadores y satisfactible con al menos dos asignaciones}\}$. Ayuda: la evaluación de una asignación sobre una fórmula booleana tarda tiempo polinomial.

...

8. Indicar si las siguientes fórmulas de correctitud son verdaderas o falsas. Justificar la respuesta en cada caso. Comentario: no hay que utilizar los métodos H y H^* , sólo aplicar las definiciones de correctitud parcial y total, y recordar que las variables son de tipo entero:

a. $\{true\}x := x + y\{x \geq y\}$

La precondition $true$ se satisface en todo estado y el programa termina siempre porque es una simple asignación. Sin embargo, tomando el estado inicial σ tal que $\sigma(x) = -2$ y $\sigma(y) = 0$, luego de ejecutar $x := x + y$ se obtiene $x = -2$. La postcondición $x \geq y$ resulta falsa, ya que $-2 \not\geq 0$.

Como existe un estado que satisface la precondition para el cual el programa termina pero la postcondición no se cumple, entonces la fórmula de correctitud es **falsa**.

b. $\{x = 0\}\text{while } y = 0 \text{ do } z := z + 1 \text{ od } \{x = 0\}$

Está claro que si $y = 0$ al inicio del programa, el programa no termina.

Si tomamos el estado σ tal que $\sigma(x) = 0$ y $\sigma(y) = 0$ tenemos que la precondition se cumple pero el programa no termina por lo tanto nunca se llega a evaluar la postcondición.

Si tomamos el estado σ tal que $\sigma(x) = 0$ y $\sigma(y) \neq 0$ tenemos que la precondition se cumple pero el programa termina sin hacer nada por no cumplirse la condición del bucle, y además se cumple la postcondición.

Por lo tanto en cualquier caso donde se cumpla la precondition y el programa termine se cumple la postcondición, por lo tanto la fórmula de correctitud es **verdadera**.

c. $\{x > 0 \wedge y > 0\}\text{while } y > x \text{ do } y := y - x \text{ od } \{true\}$

La única forma de que una fórmula de correctitud parcial sea falsa es que exista un estado inicial que satisfaga la precondition, para el cual el programa termine y la postcondición no se cumpla. Como la postcondición es true, ésta se satisface **para todo estado final**. Por lo tanto, dicho contraejemplo no puede existir y la fórmula de correctitud es **verdadera**.

9. Dado un programa S del lenguaje PLW estudiado en clase, justificar en cada caso por qué son verdaderos los siguientes enunciados:

a. Que exista algún estado σ tal que $\sigma \models p$ y $val(\pi(S, \sigma)) \models q$ no implica que $\models \{p\}S\{q\}$

La implicación no se cumple porque estamos hablando de cosas distintas. En el primer caso solo exigimos que haya UN estado σ tal que se cumple la precondition p y al ejecutar S se llega a un estado final que cumple q . En el segundo caso requerimos que para TODO estado σ tal que se cumple la precondition p , al ejecutar S se llegue a un estado final que cumpla q .

b. Que exista algún estado σ tal que $\sigma \not\models p$ y $val(\pi(S, \sigma)) \not\models q$ no implica que $\not\models \langle p \rangle S \langle q \rangle$

Por contrarrecíproca: $\models \langle p \rangle S \langle q \rangle$ no implica que no pueda existir algún estado σ tal que $\sigma \not\models p$ y $val(\pi(S, \sigma)) \not\models q$. Contraejemplo:

$\langle x = 0 \rangle skip \langle x = 0 \rangle$

1. Se cumple $\models \langle p \rangle S \langle q \rangle$ por definición.
2. Sin embargo, existe un estado σ tal que $\sigma(x) = 1$ de forma tal que tanto la precondition como la postcondition, luego de ejecutar el programa, son ambas falsas.

10. Probar con el método H la fórmula $\{p\} \text{while } p \text{ do } S \text{ od } \{q\}$, asumiendo que en H se prueba la fórmula $\{p\}S\{p\}$. Ayuda: Hay que probar con H la formula $\{p\} \text{while } p \text{ do } S \text{ od } \{q\}$ partiendo de $\{p\}S\{p\}$. Tener en cuenta: (a) a partir de $\{p\}S\{p\}$ se puede probar $\{p \wedge p\}S\{p\}$ (por la regla de consecuencia CONS), desde donde entonces luego se puede aplicar la regla de la repetición REP. (b) La aserción false implica cualquier otra.

1. $\{p\}S\{p\}$ (hipótesis)
2. $\{p \wedge p\}S\{p\}$ (por CONS en 1. ya que $p \Rightarrow p \wedge p$)
3. $\{p\} \text{while } p \text{ do } S \text{ od } \{p \wedge \neg p\}$ (aplicación de regla REP sobre 2.)
4. $\{p\} \text{while } p \text{ do } S \text{ od } \{false\}$ (por CONS en 3. ya que $(p \wedge \neg p) \Rightarrow false$)
5. $\{p\} \text{while } p \text{ do } S \text{ od } \{q\}$ (por CONS en 4. ya que como la aserción false implica cualquier otra, tenemos $false \Rightarrow q$)